

Техническое задание по Computer Vision

YOLO-детекция: бренды или дорожные знаки

DS

1 Выбор задания

Оба задания построены вокруг обучения собственного YOLO-детектора, но отличаются предметной областью и характером данных.

Обязательно: перед стартом работы важно для себя определить:

- выбранный вариант (бренды / дорожные знаки);
- планируемый список классов (3–7 классов);
- источник данных — готовый датасет, ссылки, либо план самостоятельного сбора.

2 Задание 1: Детекция логотипов брендов

2.1 Цель

Обучить YOLO-детектор, который находит логотипы выбранных 3–7 брендов в произвольной сцене: на упаковке товаров, одежде, рекламных щитах, скриншотах социальных сетей, фотографиях городской среды. На выходе — модель, способная локализовать и классифицировать логотип каждого выбранного бренда.

2.2 Источники данных

Готовые открытые датасеты:

- **LogoDet-3K** — <https://github.com/Wangjing1551/LogoDet-3K-Dataset> (3000+ брендов, 200K+ изображений, готовые bbox-аннотации).
- **FlickrLogos-32 / 47** — <http://www.multimedia-computing.de/flickrlogos/> (классический бенчмарк).
- **QMUL-OpenLogo** — <https://hangsu0730.github.io/qmul-openlogo/> (350 брендов, фокус на open-set).
- **LogosInTheWild** — датасет Fraunhofer IOSB (логотипы в естественной среде).
- **Open Images Dataset V7** — <https://storage.googleapis.com/openimages/web/index.html> (есть категория «Logo»).
- **Roboflow Universe** — <https://universe.roboflow.com/> (поиск по logo detection: десятки готовых YOLO-датасетов в нужном формате).

Самостоятельный сбор:

- **Unsplash / Pexels / Pixabay** — поиск по бренду, лицензия позволяет учебное использование.
- **Flickr Creative Commons** — <https://www.flickr.com/creativecommons/> (фильтр по лицензии CC-BY).
- **Google / Yandex Images** — допустимо для учебной работы, но с большой долей дублей и низкокачественных кадров (нужна аккуратная фильтрация).

2.3 Требования к датасету

- **Объём:** минимум 800–1500 изображений суммарно. Достижимо смещением готового датасета и собственного сбора.
- **Классы:** 3–7 брендов. Рекомендуется выбрать визуально *разные* логотипы — избегайте близких форм (например, двух похожих «галочек»), иначе модель будет путать классы и анализ выйдет неинтересным.
- **Баланс классов:** разница между самым редким и самым частым классом не более чем в 3 раза. Естественный сбор почти всегда несбалансирован — добивайте редкие классы целевым поиском.
- **Разнообразие сцен:** каждый бренд должен встречаться в разных условиях — крупно/мелко, прямо/под углом, при разном освещении, на разных фонах. Не должно быть случая, когда все картинки бренда взяты из одного источника: модель выучит фон, а не логотип.
- **Разделение на сплиты:** 70% train, 15% val, 15% test. Перед сплитом обязательно **дедубликация** (одинаковые/почти одинаковые картинки не должны попадать в разные сплиты).
- **Формат аннотаций:** YOLO — один `.txt` на картинку со строками `class_id cx cy w h` в нормализованных координатах, плюс `data.yaml` с путями и именами классов.

2.4 Советы по сборке

- **Не смешивайте «крупный план логотипа» и «логотип в сцене»** в равных пропорциях: модель должна научиться именно *детекции в окружении*, а не классификации централизованного объекта. Соотношение примерно 30/70 в пользу сцен.
- **Сложные примеры обязательны:** частично перекрытый логотип, отражения, цветовые искажения, мятая упаковка. Без них модель ломается в проде.
- **Отрицательные примеры:** 10–15% датасета должны быть картинки без целевых брендов (но с похожими объектами — другие логотипы, текст, узоры). Аннотация — пустой `.txt`-файл. Это снижает false positives.
- **Не размечайте двусмысленные случаи:** если логотип слишком размыт/обрезан и человек не уверен — пропустите, иначе шумите метками.
- Для ручной разметки используйте `label_tool.py` с семинара, либо CVAT/Label Studio/LabelImg.

3 Задание 2: Детекция дорожных знаков

3.1 Цель

Обучить YOLO-детектор дорожных знаков на кадрах с регистратора или дорожной фото-видеосъёмки. На выходе — модель, способная локализовать и классифицировать знаки выбранных 4–8 категорий в реалистичных условиях движения.

3.2 Источники данных

Готовые открытые датасеты:

- **Russian Traffic Sign Dataset (RTSD)** — <https://www.kaggle.com/datasets/watchman/rtsd-dataset> (русские знаки, ~100K кадров с регистратора).
- **German Traffic Sign Detection Benchmark (GTSDB)** — https://benchmark.ini.rub.de/gtsdb_dataset.html (классический бенчмарк).
- **Mapillary Traffic Sign Dataset** — <https://www.mapillary.com/dataset/trafficsign> (300+ классов, мировой охват, требуется регистрация).

- **Tsinghua-Tencent 100K** — <https://cg.cs.tsinghua.edu.cn/traffic-sign/> (100K кадров китайских знаков, много мелких объектов).
 - **LISA Traffic Sign Dataset** — <http://cvrr.ucsd.edu/LISA/lisa-traffic-sign-dataset.html> (США).
 - **Roboflow Universe** — поиск по `traffic signs`: множество готовых YOLO-датасетов.
- Самостоятельный сбор:**
- **Собственный регистратор / dashcam** (предпочтительно): несколько коротких поездок в разное время суток и погоду. Кадры извлекаются из видео через `ffmpeg` с шагом 0.5–1 секунда.
 - **YouTube dashcam-каналы** — кадры можно извлекать для учебной задачи.
 - **Pexels Videos / Pixabay Videos** — <https://www.pexels.com/videos/> (поиск `driving, dashcam`).

3.3 Требования к датасету

- **Объём:** минимум 500–800 кадров с аннотированными знаками. Если используется готовый датасет — взять подмножество с нужными классами.
- **Классы:** 4–8 категорий. Варианты группировки:
 - По типу: «запрещающие», «предупреждающие», «информационные», «приоритета».
 - По конкретным знакам: STOP, «уступи дорогу», ограничение скорости, пешеходный переход, «главная дорога» и т. п.

Знаки одного типа, но с разными числами (40/60/80) лучше либо объединять в один класс «ограничение скорости», либо явно разносить по классам — но не смешивать обе политики в одном датасете.
- **Баланс классов:** разница между самым редким и самым частым — не более чем в 3 раза.
- **Разнообразие условий:** день / сумерки / ночь, ясная погода / дождь / контровый свет. Без этого модель не работает в проде, даже если на валидации показывает хороший mAP.
- **Разные ракурсы и расстояния:** знак крупно (близко к камере) и мелко (за 50+ метров) — обязательно. Мелкие объекты — слабое место YOLO, к ним нужно специально готовить набор.
- **Разделение на сплиты:** 70% train, 15% val, 15% test. **Важно:** кадры из одной поездки/одного видео не должны попадать в разные сплиты — иначе случится утечка (соседние кадры почти идентичны). Сплит делать по *источнику*, а не случайно по кадрам.
- **Формат аннотаций:** YOLO (`class_id cx cy w h`) + `data.yaml`.

3.4 Советы по сборке

- **Не размечайте каждый кадр подряд:** соседние кадры почти идентичны. Шаг 0.5–1 секунда между кадрами обеспечит достаточное разнообразие без раздувания датасета.
- **Мелкие объекты:** знак, занимающий $< 10 \times 10$ пикселей на 640×640 картинке, YOLO почти не видит. Если такие случаи важны — увеличьте `imgsz` до 1024–1280 или используйте `tile`-инференс.
- **Сложные случаи обязательны:** знаки в перспективе (под углом), частично перекрытые (ветками, столбами, грязью), на засветах. Это самые частые проблемные кейсы в проде.
- **Отрицательные примеры:** длинные участки трассы / городские сцены без знаков (10–15% датасета) — снижают false positives.

- **Не размечайте знаки, которые невозможно распознать** (засветы, экстремальный motion blur, мусор на камере) — это шум, который ухудшит обучение.
- Для ручной разметки — `label_tool.py` с семинара, CVAT, Label Studio.

4 Требования к обучению (общие для обоих заданий)

4.1 Технические требования

- **Версия YOLO:** v8, v9, v10 или v11 (через библиотеку `ultralytics`).
- **Размер модели:** `nano` или `small` (`yolov8n.pt`, `yolo11s.pt` и т. п.). Большие модели для учебной задачи не оправданы.
- **Предобученные веса:** обязательно использовать (`pretrained=True`). Никакого обучения с нуля.
- **Размер входа:** `imgsz=640` как базовая; `imgsz=1024` при работе с мелкими объектами (особенно знаки на дальнем расстоянии).
- **Эпохи:** 50–150 в зависимости от объёма данных. Использовать `patience` (раннюю остановку).
- **Batch size:** максимальный, который влезает в VRAM. Для T4 на Kaggle обычно 16–32 у nano-моделей.

4.2 Что обязательно отразить в обучении

- **Аугментации:** `mosaic`, `mixup`, `hsv_h/s/v`, `degrees`, `translate`, `scale`, `flipplr` — оставить дефолтные `ultralytics` или явно подобрать. **Зеркальное отражение** (`flipplr`) для знаков и текстовых логотипов нужно либо отключить, либо уменьшить — иначе появятся «обратные» классы.
- **Учёт дисбаланса классов:** если есть редкие классы — использовать веса классов или `oversample`.
- **Learning rate schedule:** стандартный `cosine` с прогревом (`warmup_epochs=3`).
- **Логирование:** TensorBoard или встроенные `ultralytics`-логи; графики обучения — в отчёт.
- **Валидация** на `val`-сплите каждую эпоху; `test` — только в самом конце, один раз.
- **Чек на лучшем чекпоинте:** итоговые метрики снимать с `best.pt`, а не `last.pt`.
- **Воспроизводимость:** зафиксировать `seed` и записать конфигурацию обучения (`args.yaml` сохраняется автоматически в `runs/detect/<name>`).

4.3 Графики и метрики в отчёте

- Кривые **Training / Validation Loss** (`box`, `cls`, `dfl` отдельно).
- **mAP50** и **mAP50-95** по эпохам.
- **Precision-Recall curve** и **F1-Confidence curve** (генерируются автоматически в `runs/detect/<name>`).
- **Confusion matrix** на `test`-сплите.
- **Финальные метрики:** `mAP50`, `mAP50-95`, `Precision/Recall` по каждому классу отдельно.
- **FPS** на T4 (или вашем GPU) при `imgsz=640` и используемом `batch size`.
- **Примеры предсказаний:** 5–10 картинок с нанесёнными `bbox` и `confidence`. Обязательно 2–3 сложных кейса и 1–2 случая, где модель ошибается (для анализа).

4.4 Финальная демонстрация

- **Задание 1:** галерея из 10–15 тестовых изображений с предсказаниями.
- **Задание 2:** итоговое видео 30–60 секунд с регистратора с нанесёнными bbox и confidence (можно выбрать самый показательный фрагмент).

5 Требования к отчёту

- Описание выбранного варианта и обоснование выбора классов.
- Источник данных: ссылка на готовый датасет либо описание процесса самостоятельного сбора.
- Статистика по датасету: количество изображений в train/val/test, количество объектов по классам, гистограмма размеров bbox (полезно для понимания мелких объектов).
- Архитектура YOLO (название модели, число параметров).
- Конфигурация обучения: гиперпараметры, аугментации, число эпох.
- Все обязательные графики и метрики (см. раздел 4.3).
- Анализ ошибок: 2–3 проблемных кейса с гипотезой почему.
- Ссылка на финальную демонстрацию (видео или галерея изображений).

6 Формат сдачи

- **Отчёт** в формате PDF или Jupyter Notebook.
- **Архив с размеченным датасетом** (или ссылка на него, если объём >2 ГБ).
- **Код** обучения и инференса (Jupyter Notebook либо .py-скрипты).
- **Веса** обученной модели (best.pt).
- Для задания 2 — дополнительно **итоговое видео-демо**.

7 Сроки

- **Финальная сдача:** до воскресенья 31.05.2026, 23:59.